

Virtual Machine Network Simulation Project

Project Overview

Create a console-based virtual machine simulation that demonstrates distributed computing concepts. The project simulates file transfer between virtual machines using network protocols and threading.

Core Architecture

Main Components

1. **node.py** - Represents individual virtual machines
2. **network.py** - Simulates the network infrastructure and manages communications

Virtual Machine Properties

Each VM instance should have configurable parameters:

- **IP Address** - Network identifier
- **MAC Address** - Hardware identifier
- **CPU** - Processing power specifications
- **RAM** - Memory capacity
- **Storage** - Disk space
- **Network Interface** - Connection capabilities
- **Bandwidth** - Data transfer speed limits

Network Simulation Requirements

TCP/IP Protocol Stack Implementation

The simulation must demonstrate the **encapsulation/decapsulation process** through these layers:

1. **Application Layer** - File to be transferred
2. **Transport Layer** - Add port numbers and sequence numbers (creates segments)
3. **Network Layer** - Add source/destination IP addresses (creates packets)
4. **Data Link Layer** - Add source/destination MAC addresses (creates frames)
5. **Physical Layer** - Transmit as bits

File Transfer Process

1. **File Chunking** - Break large files (e.g., 100MB) into smaller segments
2. **Segment Numbering** - Each chunk gets a sequence number for proper reassembly
3. **Concurrent Processing** - Receiver starts decapsulation as soon as first bits arrive
4. **Error Checking** - Implement Cyclic Redundancy Check (CRC) for frame integrity
5. **Reassembly** - Segments may arrive out of order and need proper reconstruction

Key Features to Implement

Timing and Performance

- **System Clock** - Track transfer start/end times
- **Speed Simulation** - Calculate transfer times based on bandwidth (1Gbps vs 100Mbps)
- **Statistics Display** - Show segments sent/received, packets exchanged, transfer rates

Concurrent Operations (Threading)

- **Multiple Transactions** - Handle several file transfers simultaneously
- **Thread Management** - One thread per active operation
- **Resource Locking** - Prevent conflicts when accessing the same files
- **Socket Communication** - Use sockets to connect VMs across processes

Practical Implementation

- **Two Terminal Setup** - Run VM A in one terminal, VM B in another
- **Socket Connection** - VMs communicate using IP addresses and port numbers
- **File Transfer Protocols** - Sending machine uses specific port, receiver listens on port 21
- **Reusable Code** - Single node class with configurable parameters

Execution Flow Example

1. Start VM A (sender) and VM B (receiver) in separate terminals
2. Establish socket connection between VMs
3. VM A breaks file into chunks and begins encapsulation process

4. VM B starts receiving and decapsulating frames simultaneously
5. System tracks timing, statistics, and ensures proper file reassembly
6. Display transfer completion time and statistics

Technical Corrections and Clarifications

Clarifications on Your Notes:

- **Port 21** is typically for FTP, but you can use any available port for your simulation
- "**Lamps of ink between machines**" should be "**links between machines**"
- **Socket management** handles multiple concurrent connections, not just tracking
- The **dual operation** you described (pipelined processing) is correct - sender can be processing frame N while receiver handles frame N-1

Implementation Tips:

- Use Python's threading module for concurrent operations
- Use socket library for network communication
- Implement proper error handling for network operations
- Consider using time.time() for timing measurements
- Use threading.Lock() for resource synchronization

Success Criteria

- Demonstrate TCP/IP layer encapsulation/decapsulation
- Successfully transfer files between VMs in separate processes
- Show concurrent processing capabilities
- Display accurate timing and statistics
- Handle multiple simultaneous file transfers
- Implement proper threading and resource locking